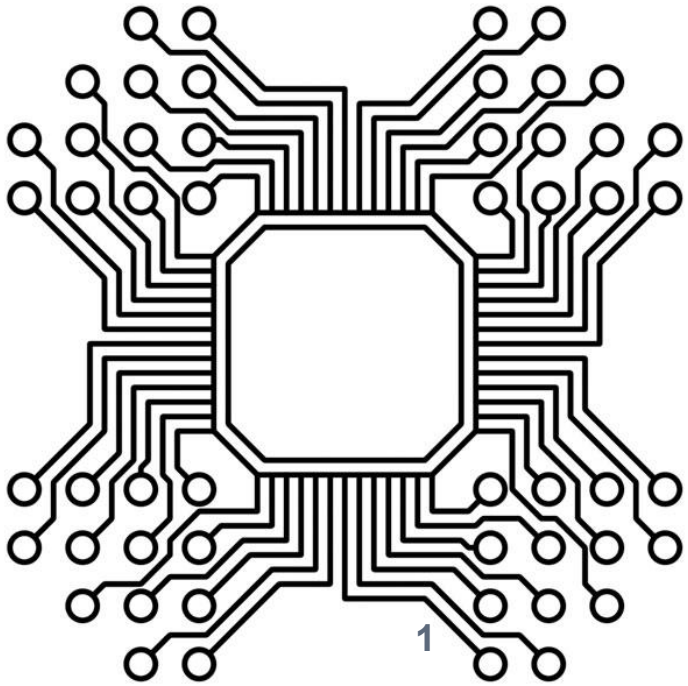
WIFI

Lecturer: Dr. Bui Ha Duc
Dept. of Mechatronics
Email: ducbh@hcmute.edu.vn



Wi-Fi



- “Wireless Fidelity”
- exchanges data wirelessly in Wireless

Local Area Network

- IEEE 802.11
  - links two or more devices using some wireless distribution method, and provides a connection through an **access point** to the wider internet.
- High Speed, wide range (50m indoor, 100m out door)
- Power consumption: chipsets 0.5-1w; wifi device 1-2w, router 4-5w,

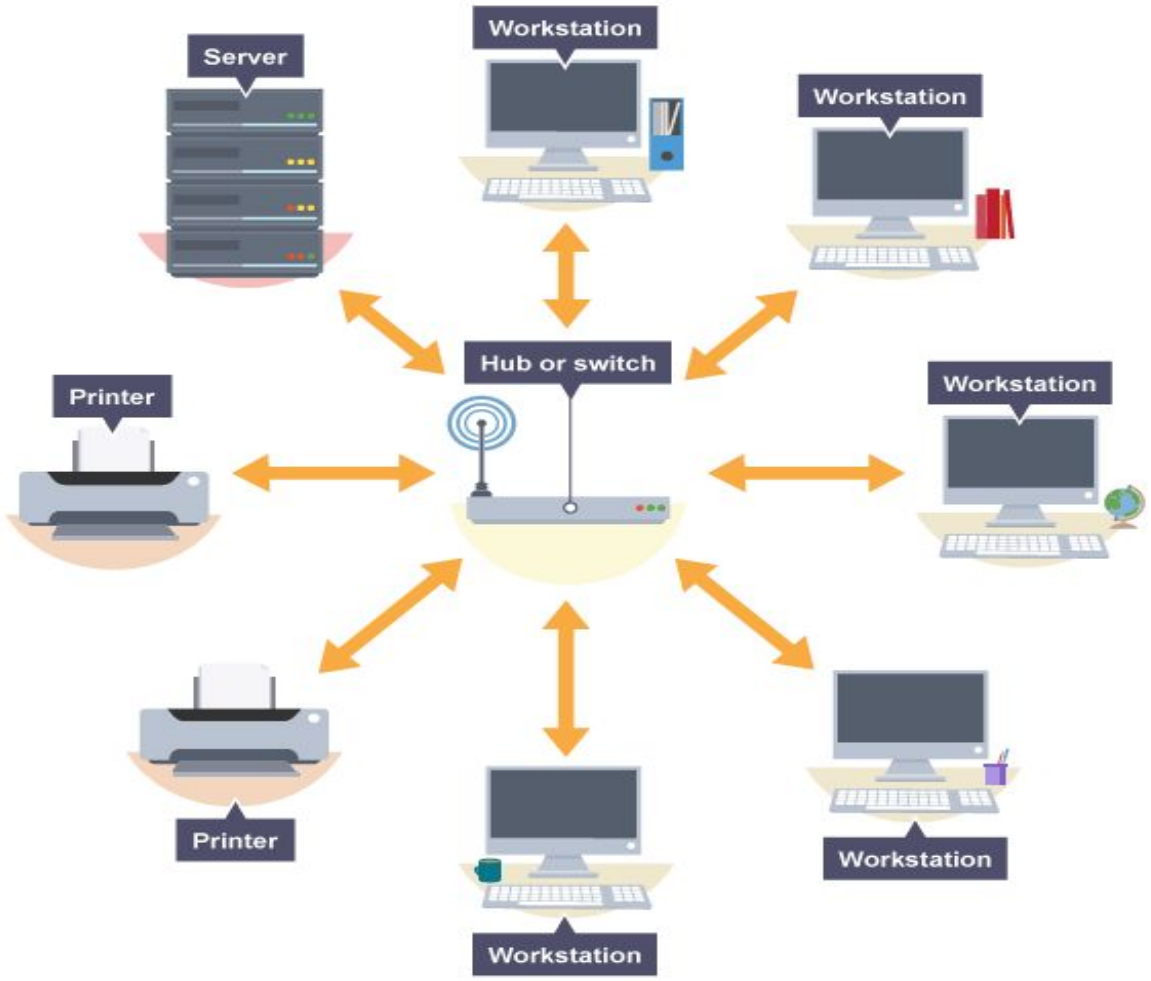
IEEE 802.11

- Set of MAC and physical layer (PHY) specifications for implementing WLAN standards (2.4, 3.6, 5GHz)
-

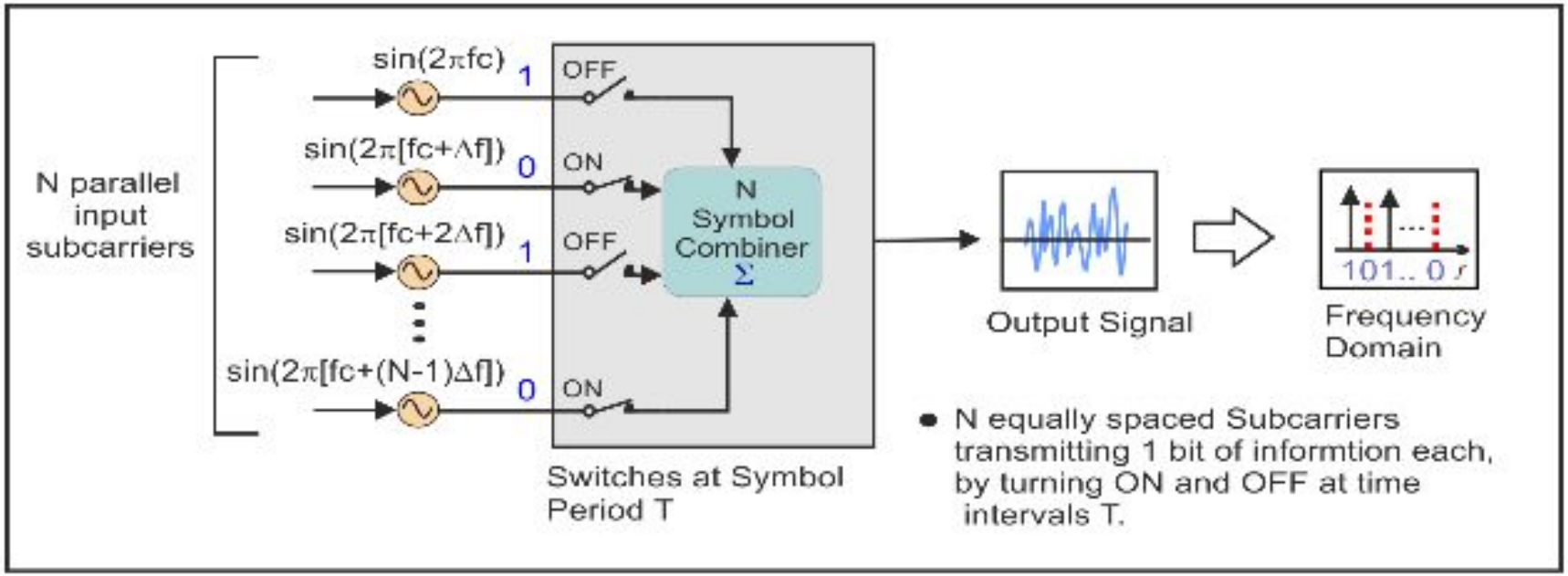
Release Year	Protocol	Frequency	Signal	Maximun data rate
1997	Legacy 802.11	2.4 GHz	FHSS or DSSS	2 Mbps
1999	802.11a	5 GHz	OFDM	54 Mbps
1999	802.11b	2.4 GHz	HR-DSSS	11 Mbps
2003	802.11g	2.4 GHz	OFDM	54 Mbps
2009	802.11n	2.4 or 5 GHz	OFDM	600 Mbps (theoretical)
2014	802.11ac	5 GHz	256-QAM	1.3 Gbps

*__OFDM__ Orthogonal frequency-division multiplexing

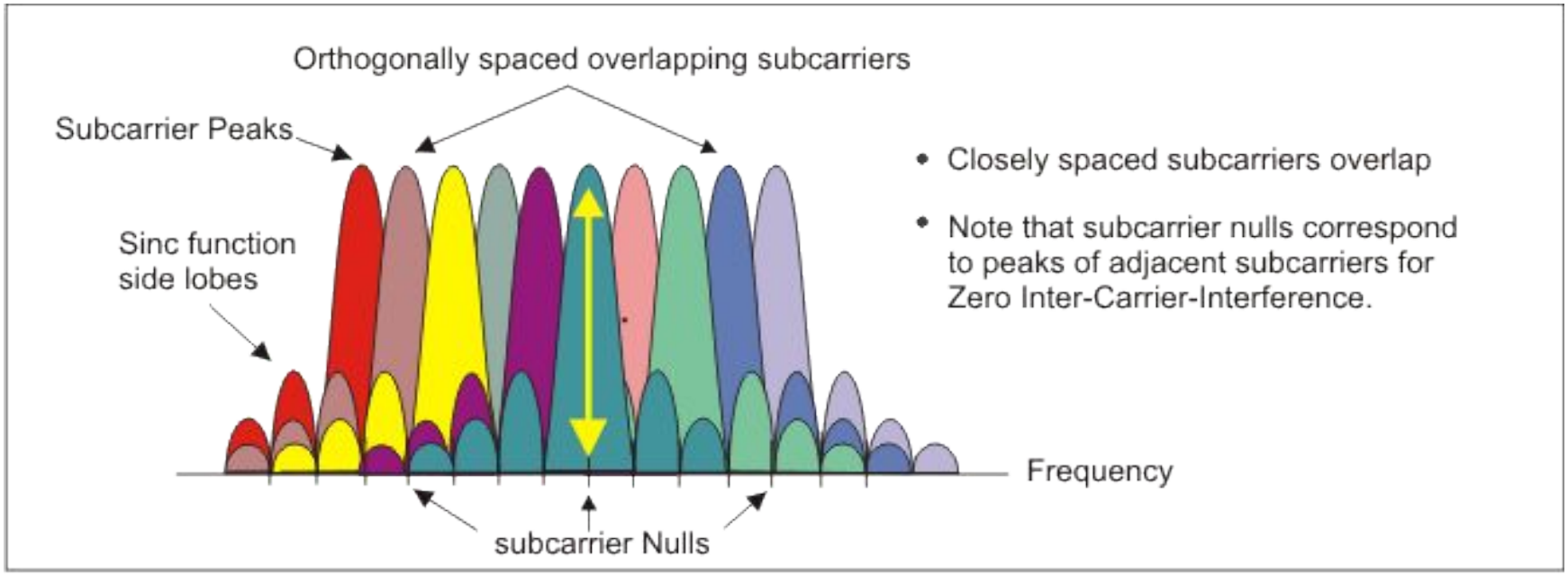
WiFi Star Network



Orthogonal frequency-division multiplexing

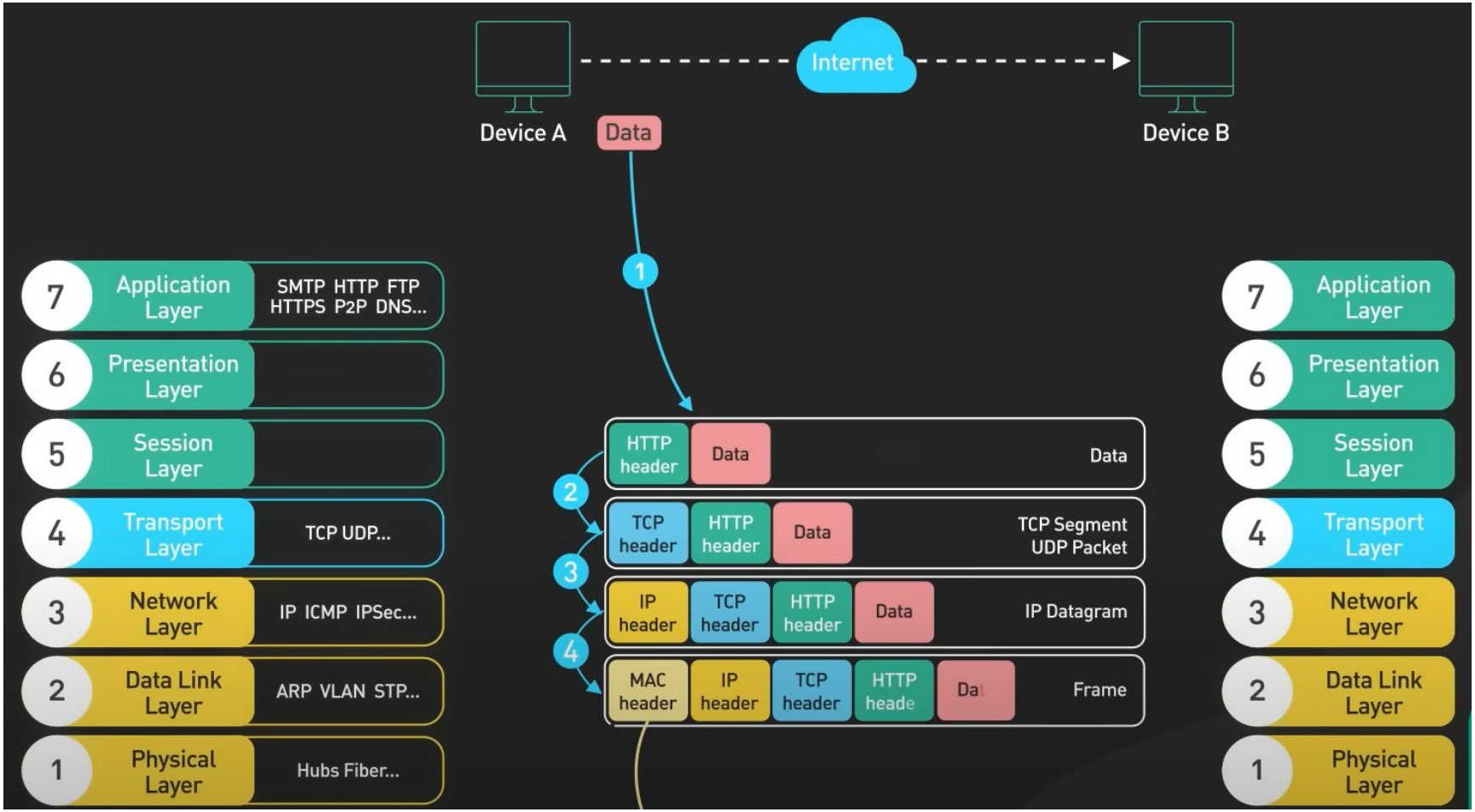


Simple OFDM Generation

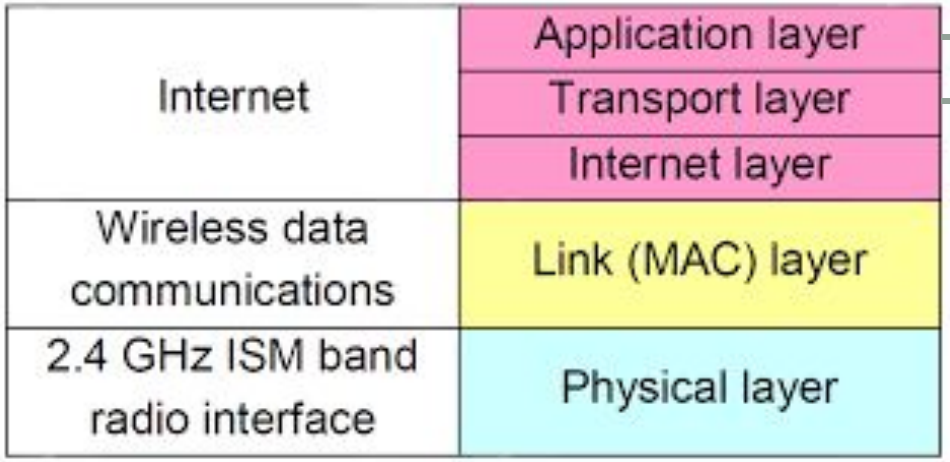


OFDM Signal Frequency Spectra

OSI Model


Internet
Device A
Data
Device B
1
7 Application Layer
SMTP HTTP FTP HTTPS P2P DNS...
6 Presentation Layer
5 Session Layer
4 Transport Layer
TCP UDP...
3 Network Layer
IP ICMP IPSec...
2 Data Link Layer
ARP VLAN STP...
1 Physical Layer
Hubs Fiber...
2
HTTP header
Data
Data
3
TCP header
HTTP header
Data
TCP Segment
UDP Packet
4
IP header
TCP header
HTTP header
Data
IP Datagram
MAC header
IP header
TCP header
HTTP heade
Da
Frame
7 Application Layer
6 Presentation Layer
5 Session Layer
4 Transport Layer
3 Network Layer
2 Data Link Layer
1 Physical Layer

WiFi Protocol Stack



Application layer: define structure of the data and how it is formatted for the software

- **HTTP/HTTPS** – web browsing and APIs.
- WebSockets – s
- **MQTT** – lightweight publish/subscribe protocol often used in IoT over Wi-Fi.
- **CoAP** – optimized for IoT devices, runs over UDP.

Transport Layer: managing how data moves end-to-end between two devices

- Treat all data as stream of bytes
- **TCP and UDP**

WiFi Protocol Stack

Internet/Network Layer

- Assigns unique identifiers - IP addresses - to every device on a network
- Move data from source IP to destination IP effectively
- If a packet is too large, the internet layer splits it into smaller fragments and reassembles them at the destination
- Major protocols: IPv4 and IPv6
  - IPv4: 32-bit address (e.g. *192.168.1.1*)
  - IPv6: 128-bit address (e.g. *2001:0db8:85a3:0000:0000:8a2e:0370:7334*)

WiFi Protocol Stack

Transport Layer

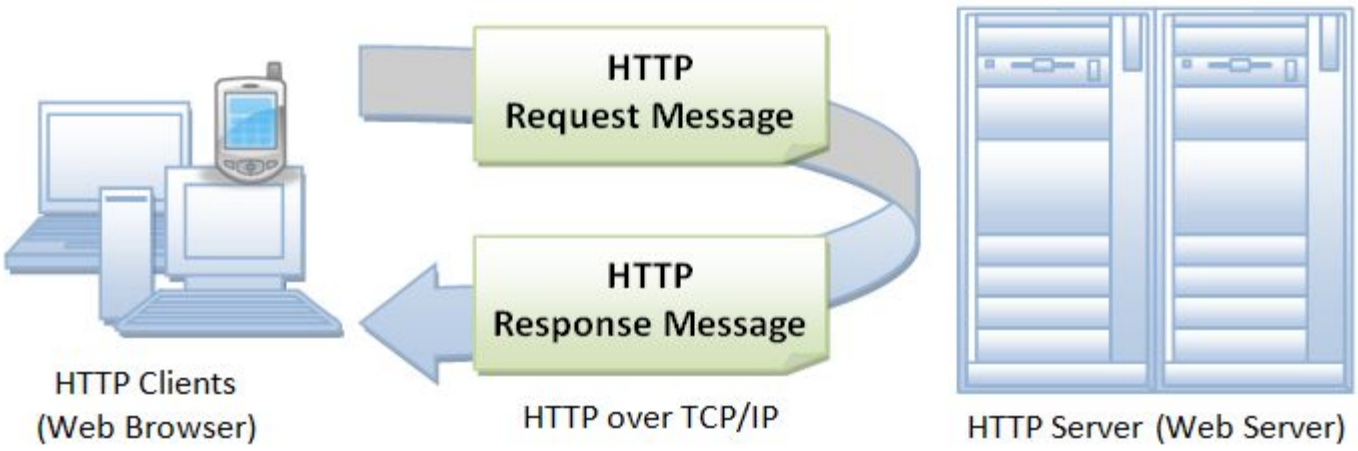
- Manage how data moves end-to-end between two devices
- Uses port numbers to ensure incoming data is routed to the exact application
- Treat all data as stream of bytes
- Major protocols: TCP and UDP
- Transport layer is usually managed by the OS kernel

WiFi Protocol Stack

- **Application layer**
- Define structure of the data and how it is formatted for the software
- **Major Protocols:**
  - **HTTP/HTTPS** – web browsing and APIs.
  - **WebSockets** – providing full-duplex communication channels over a single TCP connection
  - **MQTT** – lightweight publish/subscribe protocol often used in IoT over Wi-Fi.
  - **CoAP** – optimized for IoT devices, runs over UDP.
  - **DDS** – peer-to-peer, realtime communication protocol

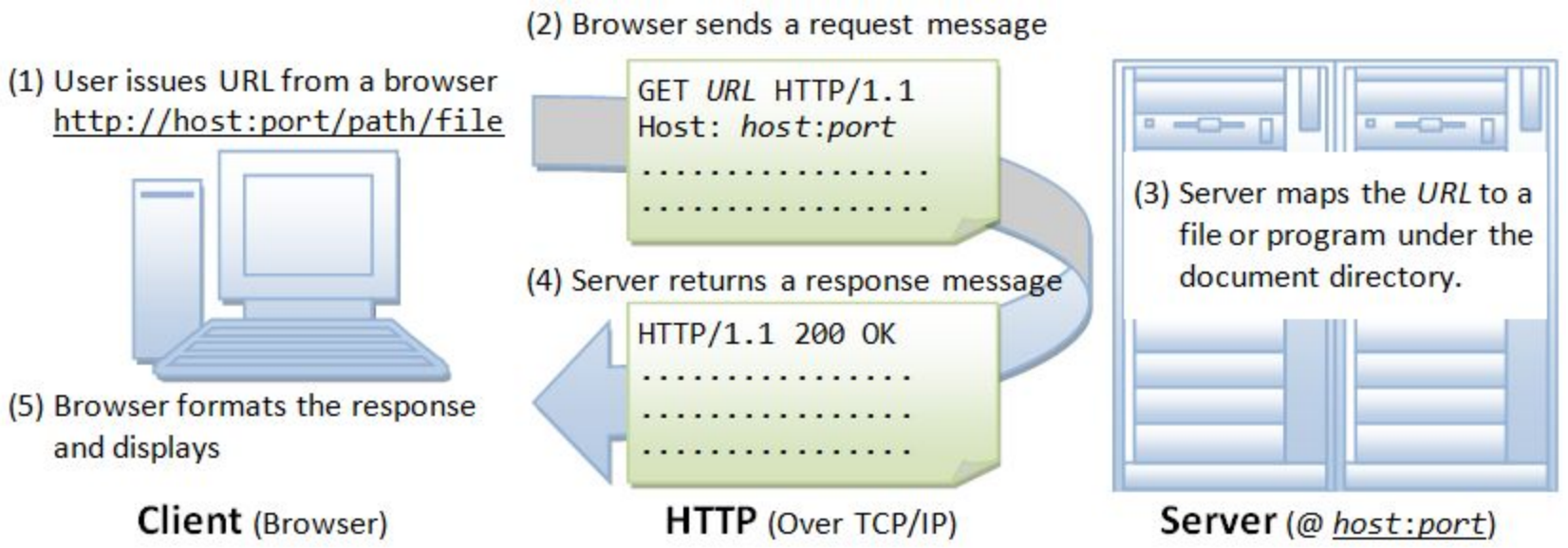
HyperText Transfer Protocol (HTTP)

- HTTP is an *asymmetric request-response client-server* protocol
- HTTP is a stateless protocol



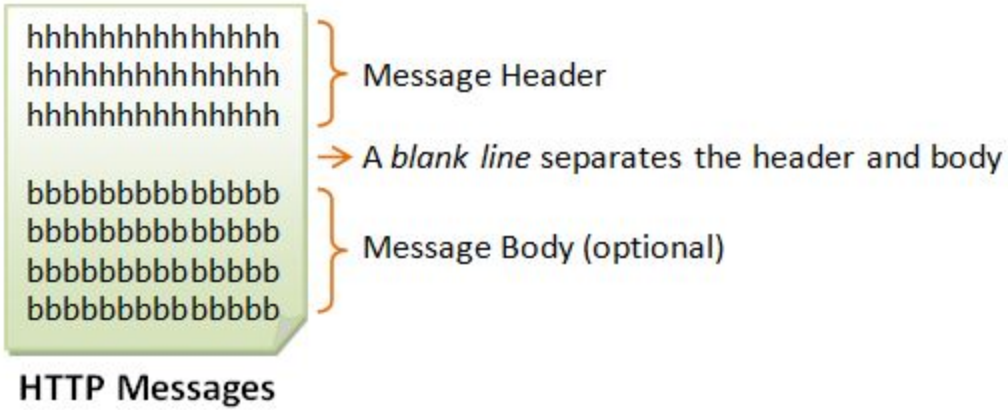
http://www.ntu.edu.sg/home/ehchua/programming/webprogramming/http_basics.html

HyperText Transfer Protocol

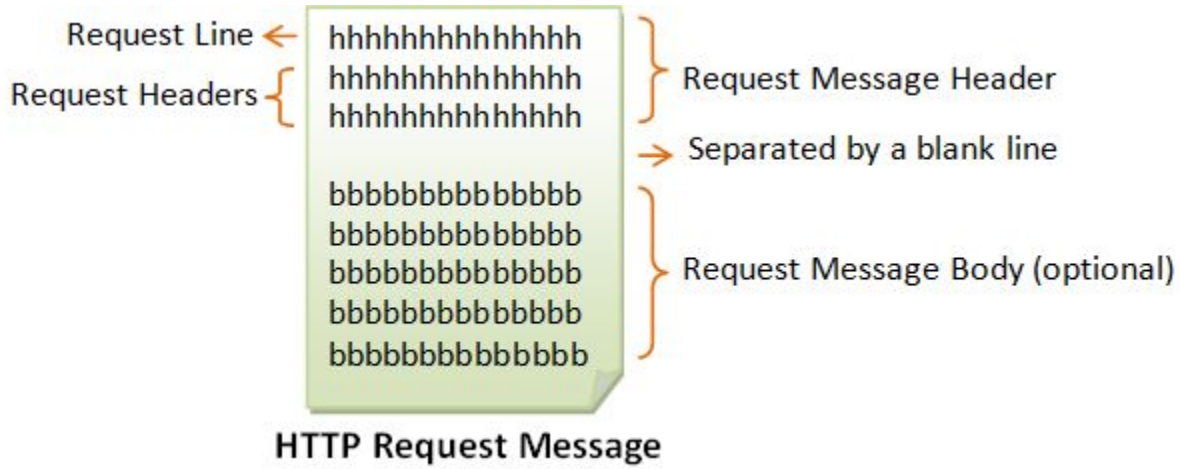


HTTP Messages

- HTTP client and server communicate by sending **text messages**
- An HTTP message consists of a *message header* and an optional *message body*, separated by a *blank line.*



HTTP Request Message



- The request line has the following syntax:

  request-method-name *request-URI* *HTTP-version*

  - *request-method-name*: request methods, e.g., GET, POST, HEAD, and OPTIONS.
  - *request-URI*: specifies the resource requested.
  - *HTTP-version*: Two versions are currently in use: HTTP/1.0 and HTTP/1.1.

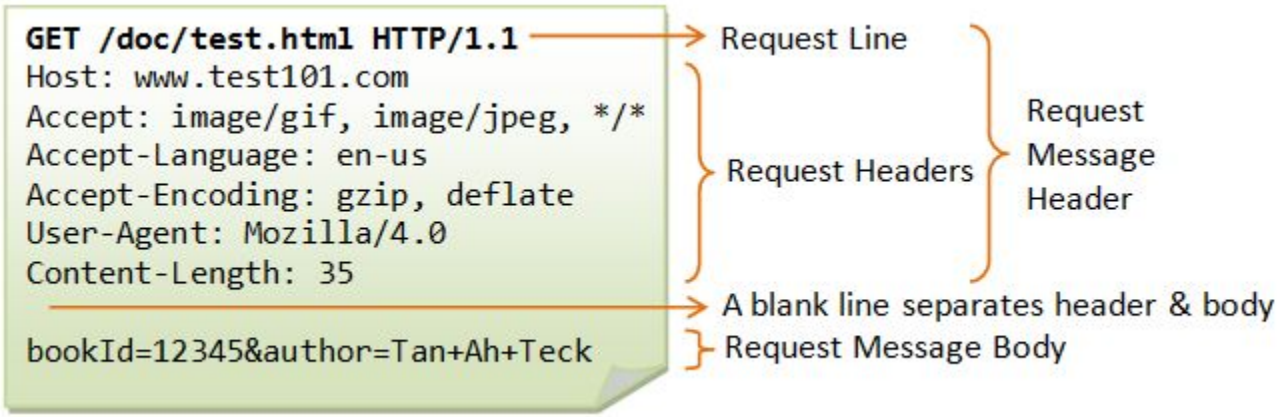
HTTP Request Message

- The request headers Syntax:

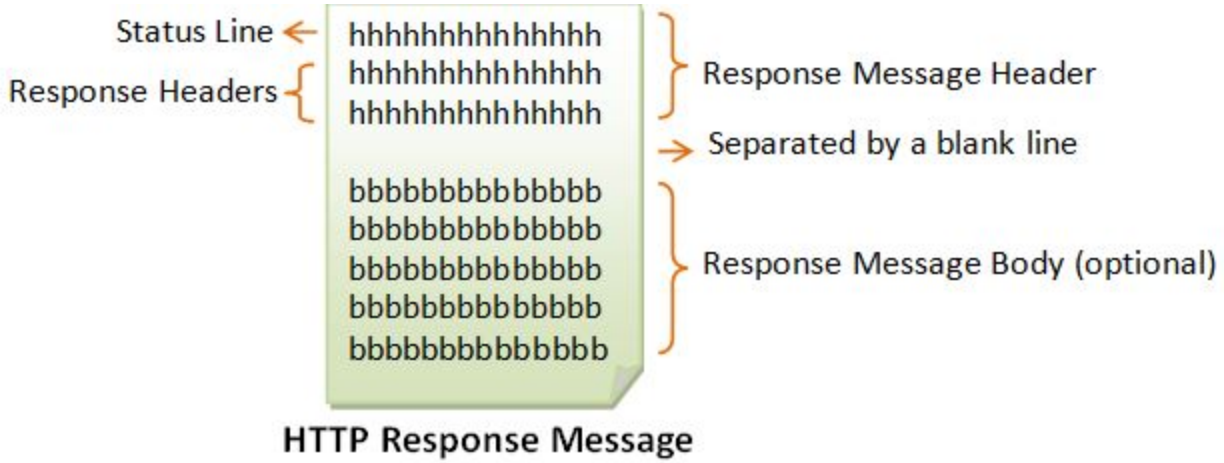
request-header-name: *request-header-value1*, *request-header-value2*, ...

- request message body: contain user inputs

  name1=value1&name2=value2...



HTTP Response Message



- The status line has the following syntax:

  HTTP-version status-code reason-phrase

  - *HTTP-version*: The HTTP version used in this session. Either HTTP/1.0 and HTTP/1.1.
  - *status-code*: a 3-digit number generated by the server to reflect the outcome of the request. (200, 403, 404…)
  - *reason-phrase*: gives a short explanation to the status code

HTTP Response Message

- The response headers syntax:

```
response-header-name: response-header-value1, response-header-value2, ...
```

- Response message body: return value/ website

